



# Function as a Service for HPC

Duccio Rocca<sup>1</sup>, Nicholas Tyler<sup>2</sup>

<sup>1</sup>San Francisco State University, <sup>2</sup>Lawrence Berkeley National Laboratory



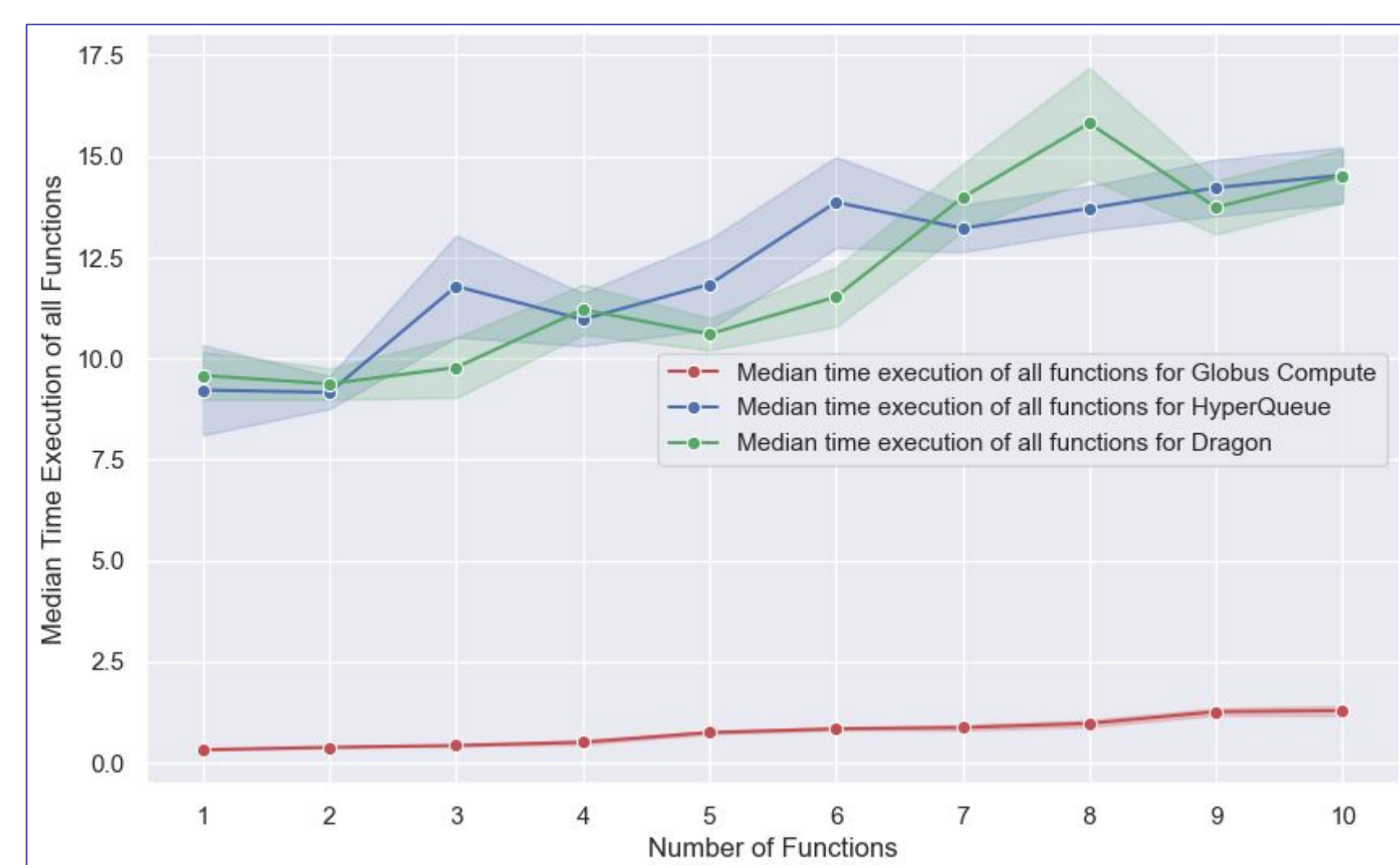
Office of Science



## Tools Comparison

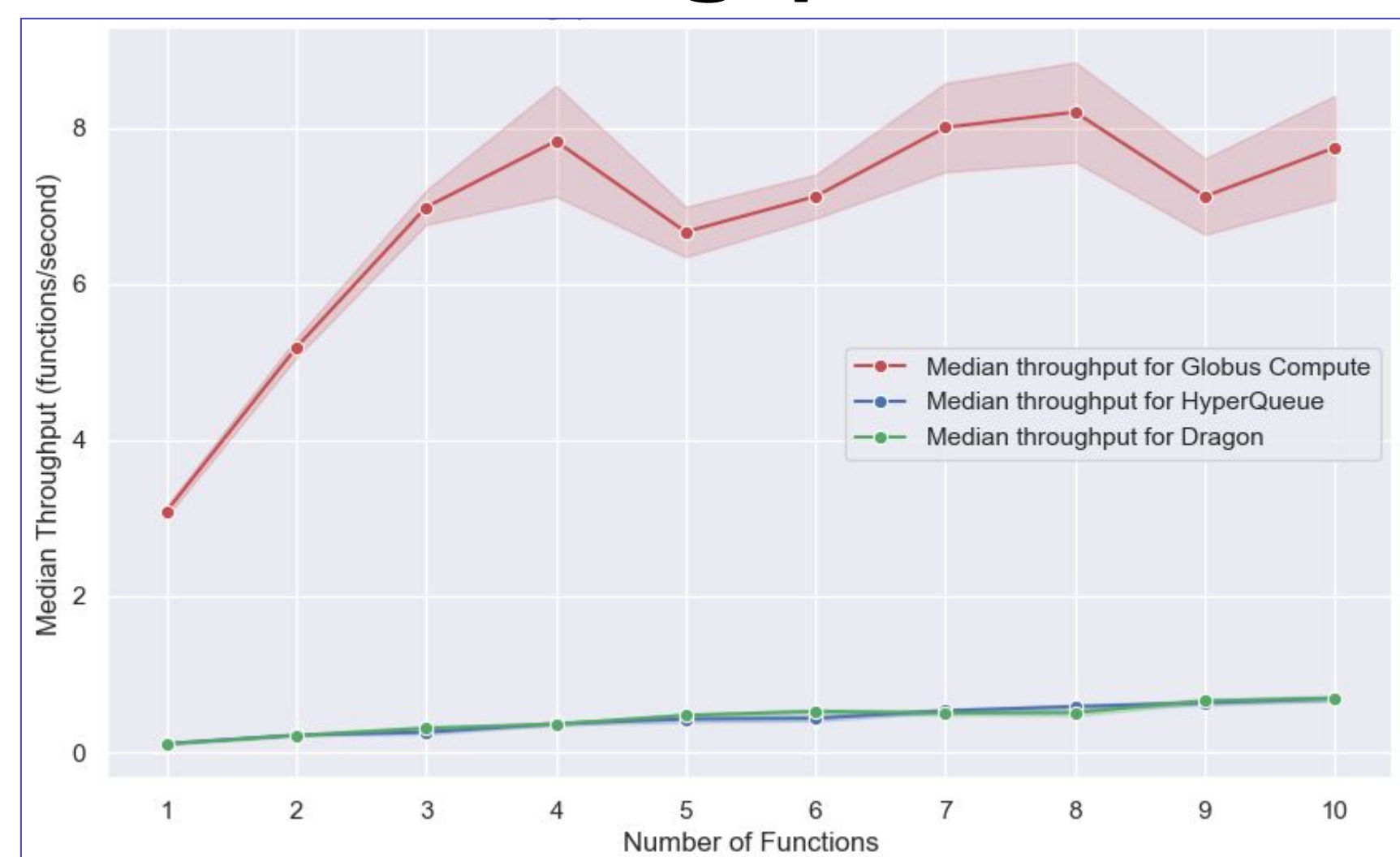
For our initial experiments, we executed PyTorch inference on a GPU node using all three tools. We started with one function and increased up to ten running concurrently, measuring execution time and calculating throughput. It is worth noting that we wrote the function once and ran it on every tool without modification.

### Execution Time



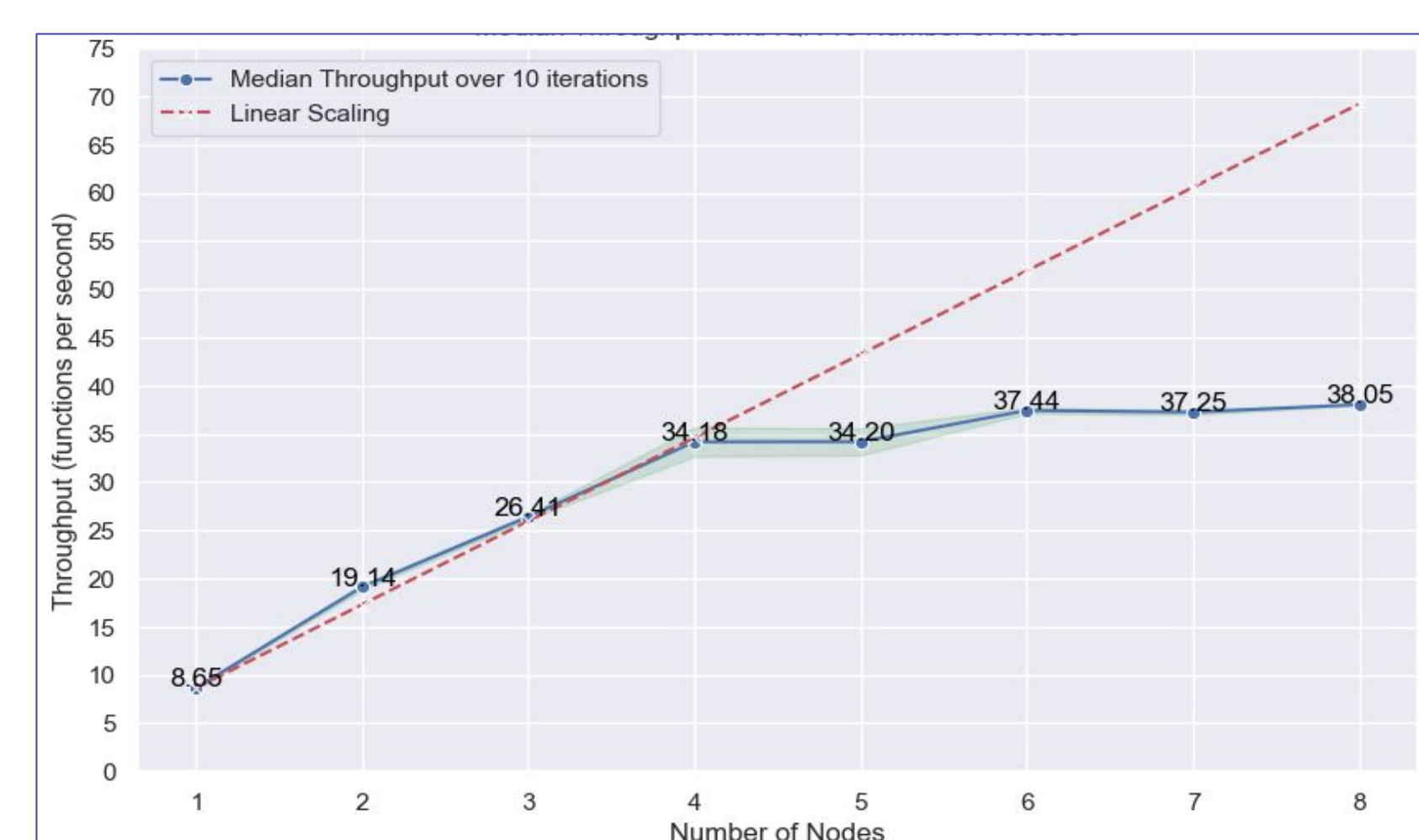
A comparison of the median execution time (in seconds) across 50 iterations starting from a single function, up to ten concurrent functions, shows that GC has significantly better performance across the entire range of functions. The key difference resides in how processes are started for each function.

### Throughput



The median throughput over 50 iterations demonstrates that GC consistently outperforms other tools and scales effectively with the number of GPUs, ranging from 1 to 4 and 5 to 8.

## Scaling Globus Compute



Scaling the GC deployment of PyTorch code across multiple nodes demonstrates linear behavior up to four nodes, after which it plateaus at approximately 38 functions per second.

## Abstract

This study investigates the performance of Functions as a Service (FaaS) tools in a high-performance computing (HPC) environment. We evaluate their runtime efficiency and throughput capabilities through practical experiments. Specifically, we deploy a PyTorch inference code using a ResNet model for image classification. Based on initial experimental results, we identify the most performant FaaS tool and further assess its throughput by scaling through increasing functions and nodes. Additionally, we deploy a real-world code with a GPU-intensive workload to measure its performance in terms of throughput.

## HPC vs FaaS

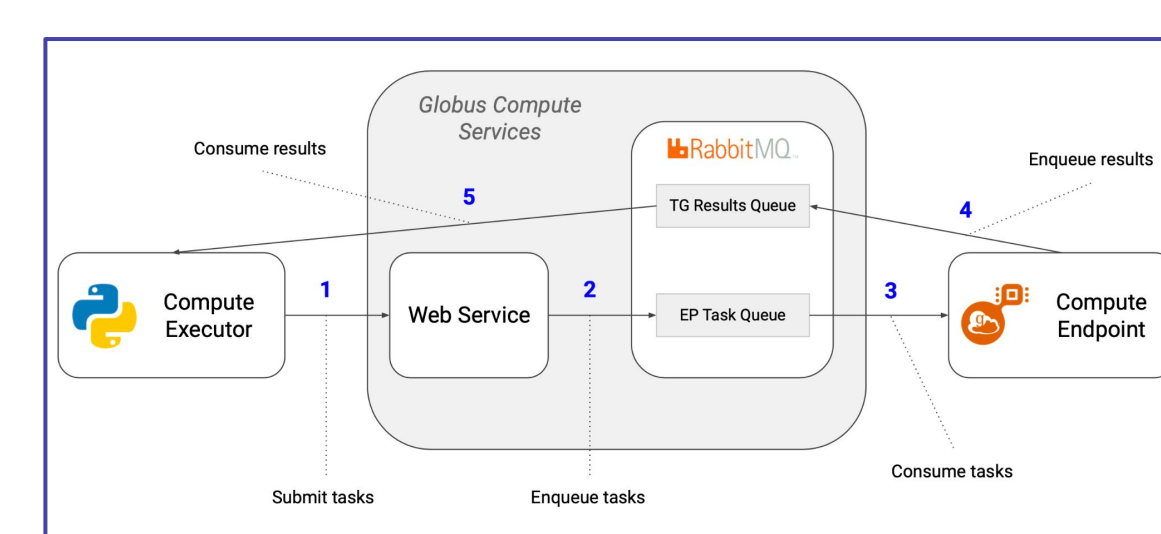
Although HPC and FaaS are all used in scientific computing, their workload characteristics are substantially different.

- In **HPC**, a job is assigned a requested number of nodes for a given period of time and has to manage these resources on its own.
- The provider has no job-level responsibility; instead, they maximize the overall utilization of the cluster expressed as a fraction of nodes allocated to jobs over time.
- In **FaaS**, users deploy individual functions, and providers handle their execution scheduling.
- Users invoke functions individually rather than managing entire jobs, with providers allocating resources per invocation.
- FaaS simplifies development and maintenance offering a fire-and-forget execution model.

## Deployed Tools

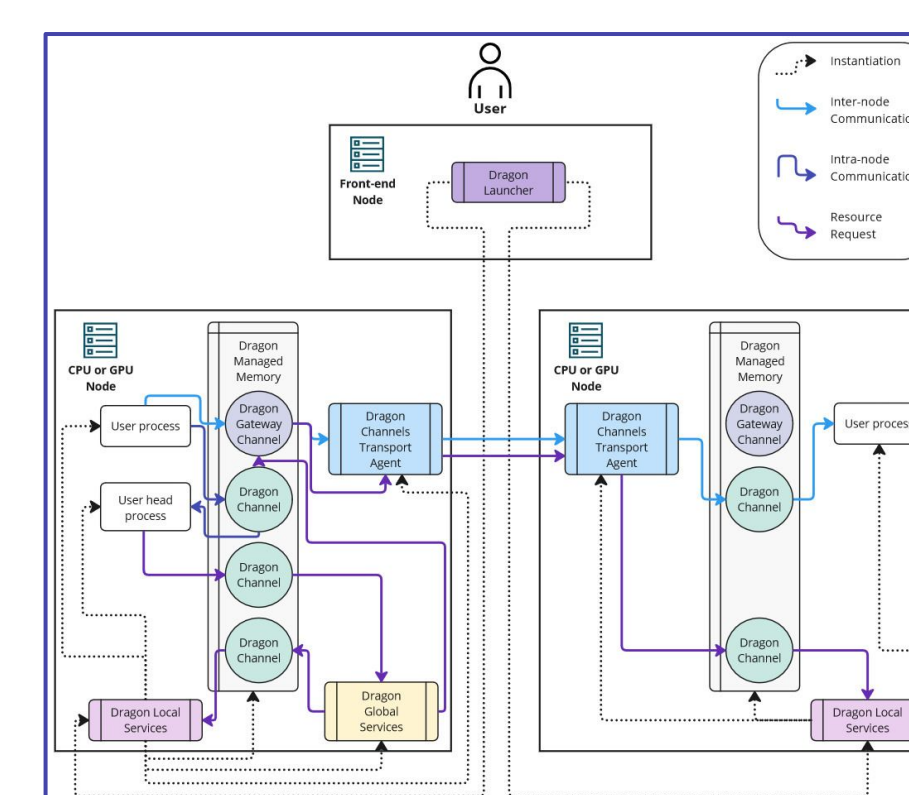
### Globus Compute (GC)

A distributed FaaS platform for high performance remote function execution. Implemented by researchers at Argonne national Lab and University of Chicago.



Source: Josh Bryan slides at GlobusWorld 2024

### Dragon

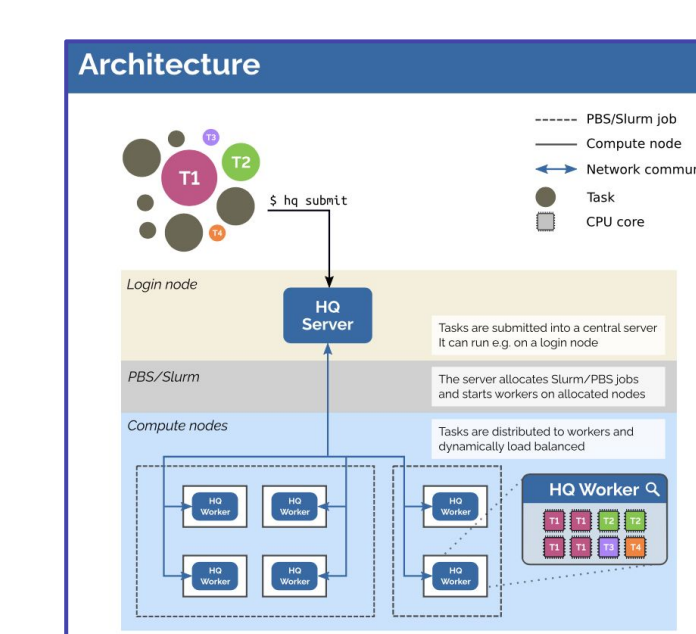


Source: dragonhpc docs

Dragon is a composable distributed run-time for managing dynamic processes, memory, and data at scale through high-performance communication objects.

### HyperQueue (HQ)

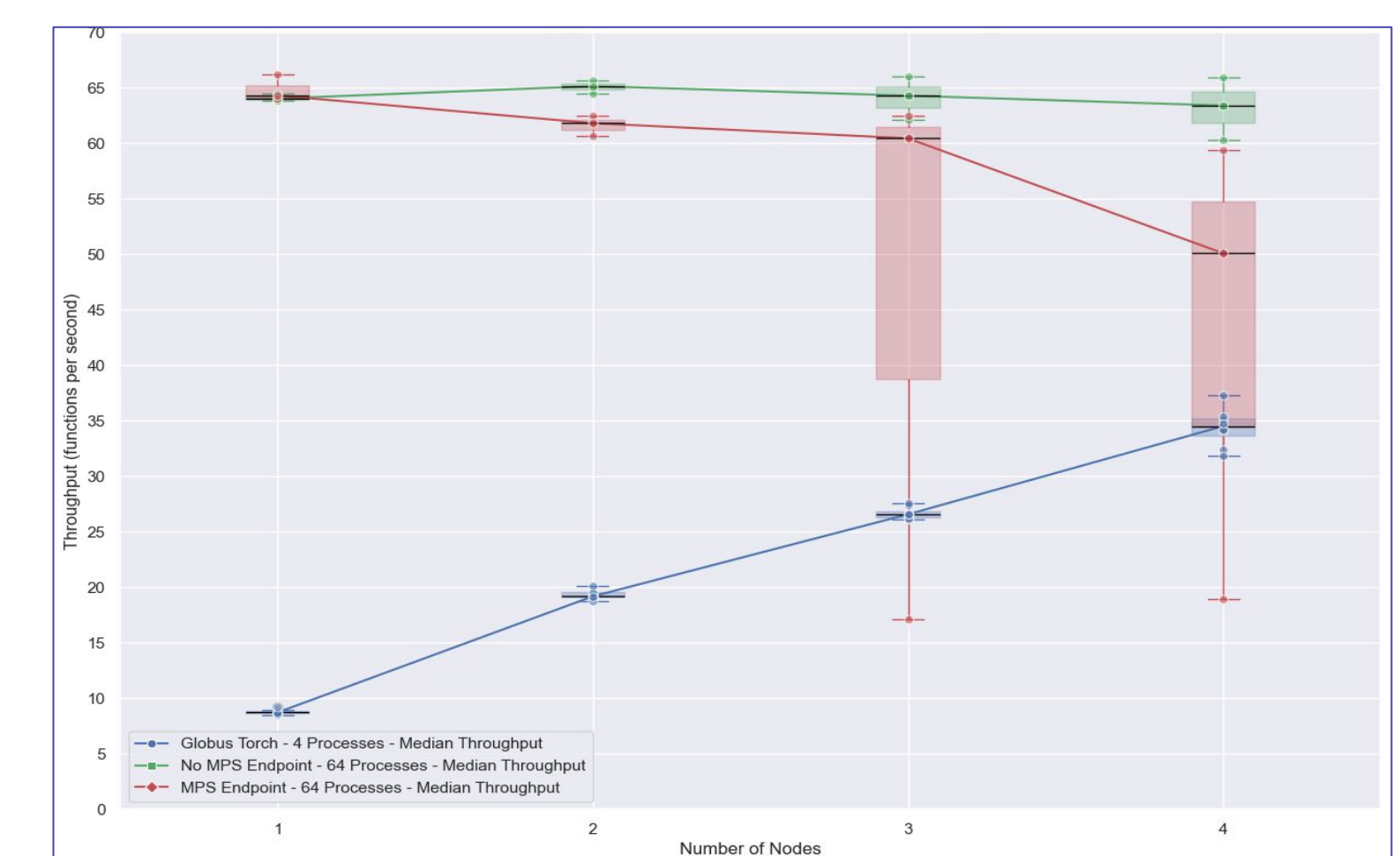
HyperQueue is a task executor that simplifies executing large workflows on HPC clusters. It automates task submission and resource management, dynamically balancing tasks across all allocated resources.



Source: SC21 poster <https://def.it4i.io>

## Oversubscribing GPUs

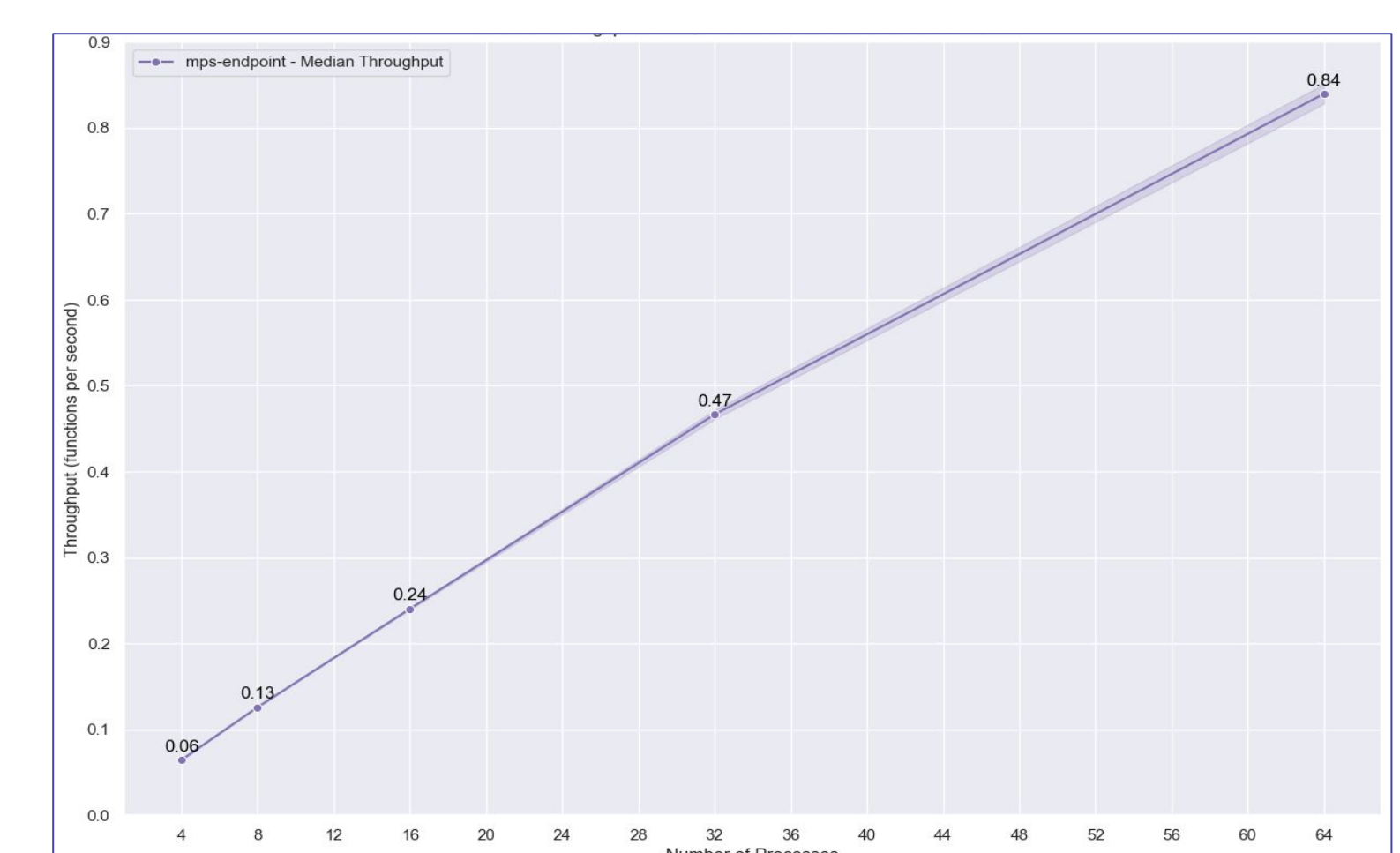
We oversubscribed the GPUs to increase the number of processes running in parallel on each GPU, scaling up to the maximum number of processes (64 per node) on multiple nodes. We compared the performance results achieved with and without the Multi-Process Service (MPS).



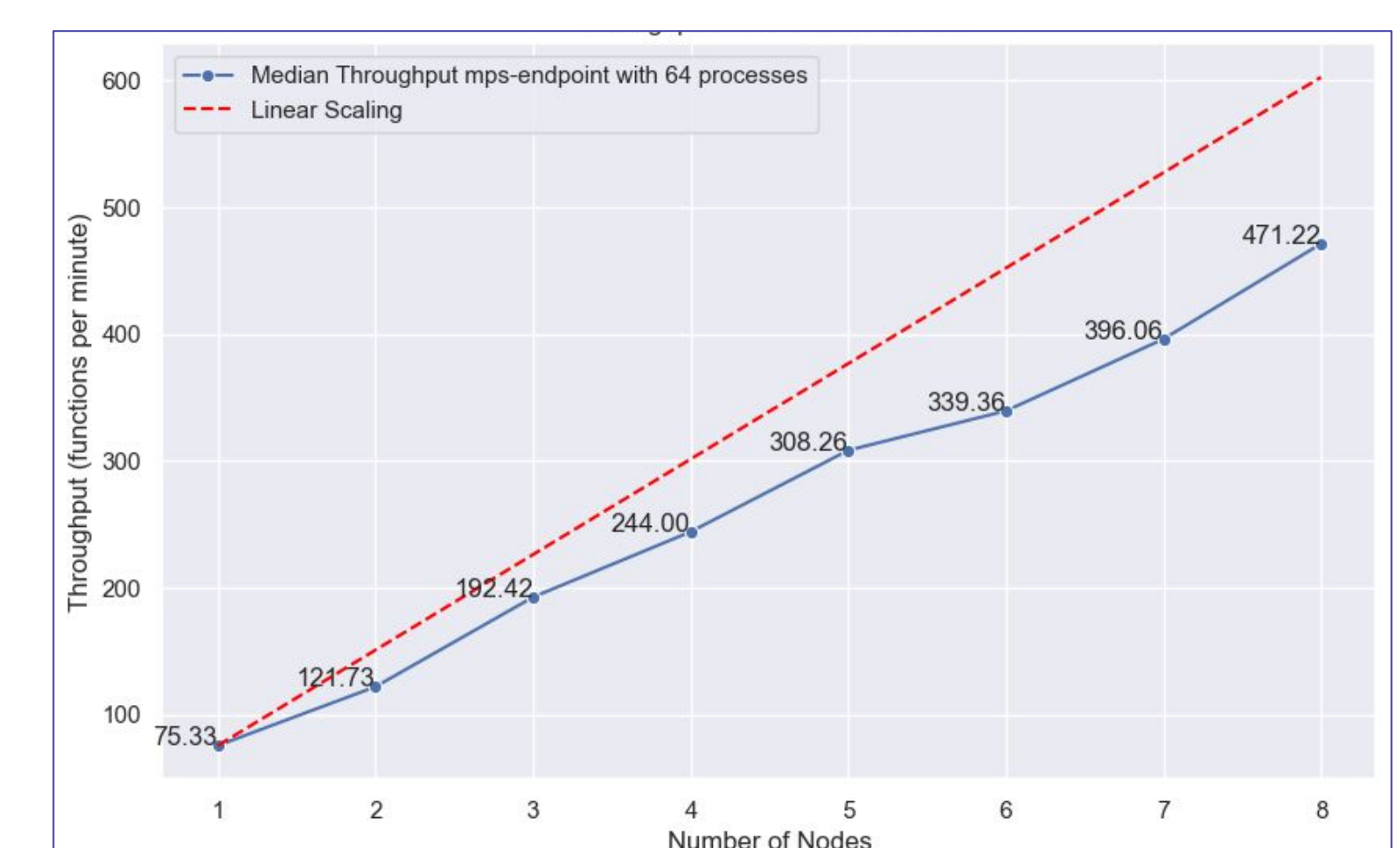
Both MPS and No-MPS configurations exhibited improved performance with 64 parallel processes compared to four (without oversubscription). However, neither configuration demonstrated effective scalability with an increased number of nodes.

## Scaling real-world workload

We deployed a real-world workload, specifically the Ion Orbiter code which simulates particle trajectories and determines their hit locations on the tokamak wall, using GC. We scaled the deployment by first doubling the number of processes and then increasing the number of nodes, measuring throughput at each step.



The observed scaling with GC and MPS is nearly linear, with throughput doubling as the number of parallel processes on the GPU-node doubles.



We applied strong scaling to the 64-process configuration by increasing the nodes while keeping the number of functions (1000) fixed. Although the scaling is non-linear, the throughput demonstrates satisfactory scalability.

## Acknowledgments:

We thankfully acknowledge Andrew Naylor's support

